

**iSpring**

Платформа для корпоративного обучения

64,97 Рейтинг

218 Подписчики

[Подписаться](#)**pavelfargo**

15 минут назад

Скриншотное тестирование: практические советы из реального проекта

Простой 8 мин 607

Блог компании iSpring, Тестирование IT-систем*, Тестирование веб-сервисов*

[Тutorial](#)

Меня зовут Павел Попов, я инженер по автоматизации тестирования в компании iSpring. В этой статье поделюсь опытом использования скриншотных тестов: расскажу, какие задачи мы решаем с их помощью, с какими проблемами столкнулись и какие практики помогают нам поддерживать такие тесты в рабочем состоянии.

В нашем арсенале более 2500 скриншотных тестов, поэтому многие выводы сделаны на основе длительного опыта эксплуатации.

Сначала разберёмся, как работают такие тесты, а затем перейдём к практическим рекомендациям.

Что такое скриншотные тесты

Скриншотные тесты — это способ проверки интерфейса приложения путём сравнения изображений.

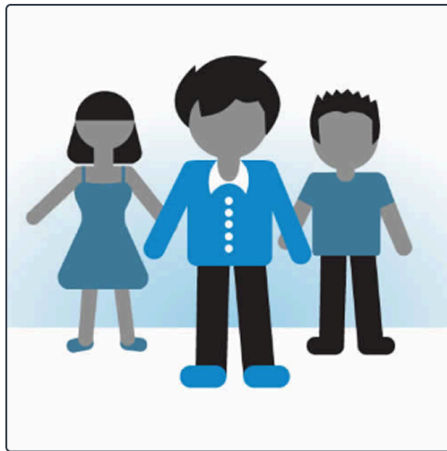
Принцип работы достаточно простой: система сравнивает два изображения интерфейса — эталонное и актуальное.

- Эталонный скриншот создаётся один раз и проходит ручную проверку. Тестировщик убеждается, что интерфейс отображается корректно, после чего изображение сохраняется и используется как ожидаемый результат теста.
- Актуальный скриншот создаётся автоматически во время выполнения теста. Он отражает текущее состояние интерфейса и используется как фактический результат.

Для наглядности используем изображения и примеры сравнения с ресурса [Resemble.js](#). Они хорошо иллюстрируют принцип работы скриншотных тестов.



Эталонный скриншот



Актуальный скриншот

Сравнение изображений

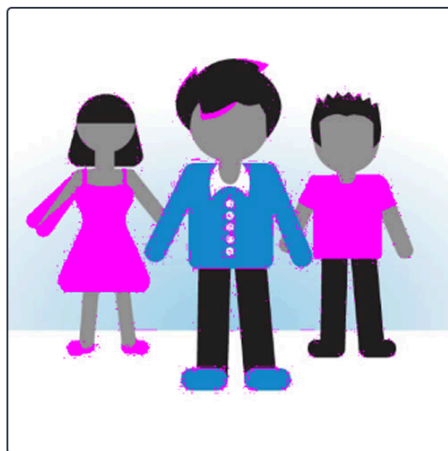
При сравнении двух изображений вычисляется коэффициент различия.

- 0 — изображения полностью совпадают.
- 1 — изображения полностью различаются.

Например, между эталонным и актуальным скриншотами выше есть различия, которые почти незаметны для человека.

Визуализация различий

Если изображения не совпадают, система формирует третье изображение, на котором выделены различия.



В нашем примере различия следующие:

- изменилось положение руки у персонажа слева;
- немного изменилось положение волос у центральной фигуры;

- изменился цвет одежды у боковых персонажей.

Важно понимать: самый простой и распространённый способ сравнения скриншотов — попиксельный. В этом случае изображения сравниваются пиксель к пикселю, и фиксируются даже минимальные отличия.

Мы используем именно этот подход. При этом существуют и более продвинутые методы сравнения.

Преимущества скриншотных тестов

Такой подход имеет несколько сильных сторон.

1. Надёжная защита от визуальных регрессий

Тест способен обнаружить любое изменение интерфейса — даже то, которое сложно заметить вручную.

2. Простая проверка

Вместо множества отдельных проверок (цвет, размер, положение элементов) достаточно сравнить одно изображение с эталоном.

3. Наглядный отчёт

В отчёте сразу видно, где именно произошло изменение, поскольку различия подсвечиваются на изображении.

Как мы используем скриншотные тесты

В нашей команде такие тесты решают две основные задачи.

1. Контроль интерфейса большого количества страниц

У компании iSpring есть несколько сайтов с информацией о продуктах и компании. Всего их 14, и они размещены на разных серверах.

У каждого сайта могут быть свои атрибуты, отличные от остальных, например: свой язык или собственные настройки оформления.

После каждого обновления необходимо убедиться, что интерфейс страниц не сломался.

Проверять всё вручную крайне трудоёмко из-за большого количества страниц, поэтому мы используем скриншотные тесты. Они автоматически проверяют:

- оформление страниц;
- корректность отображения текстов;
- структуру интерфейса.

2. Проверка визуального контента

В наших продуктах есть редакторы для создания обучающих материалов. Один из них — редактор лонгридов.

Контент в таком редакторе собирается из блоков:

- заголовки;
- текстовые параграфы;
- изображения;
- интерактивные элементы.

Всего используется около 30 типов блоков.

Функциональные тесты не могут проверить корректность визуального отображения таких элементов. Поэтому здесь также применяются скриншотные тесты.

Немного о запуске тестов

Для скриншотных тестов мы используем браузер Chrome как наиболее популярный браузер. Экземпляры браузеров запускаются в контейнерах, управлением которых занимается selenoid. Каждый браузер живет своей независимой жизнью от других браузеров. Это позволяет параллелизировать прогоны тестов, что хорошо ускоряет процесс тестирования.

Тесты запускаются на страницах собранного приложения без подмены данных.

Для нас важно, чтобы элементы отображались именно там, где это задумывалось: с реальными отступами, стилями и с настоящим текстом.

Проблема: хрупкость скриншотных тестов

На практике стало ясно, что такие тесты достаточно чувствительны. Даже изменение в один пиксель может привести к падению теста.

Когда тест падает, возникает вопрос: это ошибка интерфейса или корректное изменение приложения?

Иногда причиной могут быть неожиданные факторы:

- обновление браузера;
- задержки при загрузке страницы;
- различия сглаживания шрифтов в разных операционных системах.

Такие ситуации приводят к нестабильности тестов.

Чтобы уменьшить количество ложных срабатываний, внутри команды мы сформировали несколько правил, которых придерживаемся. Далее расскажу подробно о каждом из них.

Правила работы со скриншотными тестами

1. Используем единое окружение для создания скриншотов

Изначально эталонные скриншоты создавались на рабочих компьютерах тестировщиков, а тесты запускались на удалённых браузерах. Тогда мы столкнулись с ситуацией, когда локально тест проходил успешно, а на удалённом браузере он падал.

Мы пришли к выводу, что эталоны должны создаваться в том же окружении, где выполняются тесты. Следовательно, это вылилось в правило: **эталонные скриншоты создаем на удалённых браузерах.**

Это позволяет устранить различия, связанные с:

- особенностями рендеринга;
- различиями DPI;
- особенностями операционных систем.

2. Убираем лишние элементы страницы

На странице могут быть элементы, которые мешают тестированию:

- баннер согласия об использовании cookies;
- всплывающий чат поддержки;
- рекламные блоки;

- динамический контент (видео, аудио плееры).

При их появлении происходит перекрытие нужного интерфейса или изменяемый, от теста к тесту, визуальный шум в изображении.

Такие элементы лучше всего **скрыть** или **удалить из отображения**.

Рассмотрим простой пример, на котором продемонстрирую оба подхода.

У нас есть HTML-страничка с элементами:

Если к синему квадрату применить CSS-свойство `visibility: hidden;` элемент останется в структуре страницы, но станет невидимым — это скрытие элемента.

Если к элементу применить CSS-свойство `display: none;` элемент полностью исключится из отображения, а соседние элементы сдвигнутся.

Данное правило позволяет убрать лишний шум со странички и получить более стабильные тесты.

3. Отключаем CSS-анимации

Проигрывание анимации создает определенные ограничения в скриншотном тестировании. Они могут запускаться непредсказуемо в разное время, что будет приводить к различиям между скриншотами.

Чтобы избежать этого, перед созданием скриншота мы **отключаем анимации** через CSS следующим образом:

```
*, *::before, *::after {
  -o-transition-property: none !important;
  -moz-transition-property: none !important;
  -ms-transition-property: none !important;
  -webkit-transition-property: none !important;
  transition-property: none !important;
  -webkit-animation: none !important;
  -moz-animation: none !important;
  -o-animation: none !important;
  -ms-animation: none !important;
  animation: none !important;
  caret-color: transparent !important;
}
```

Объяснить с  SourceCraft

После этого CSS-анимации не воспроизводятся до перезагрузки страницы, и тест может сделать корректный снимок интерфейса.

4. Используем маскирование

Иногда интерфейс содержит данные, которые меняются при каждом запуске теста:

- случайные цвета в интерфейсе;
- автоматически сгенерированный текст;

Это приводит к постоянным различиям между изображениями. Для решения этой проблемы применяется **маскирование**.

Данный подход используется для исключения определенных областей изображения из процесса сравнения изображений. Это необходимо, чтобы избежать ложноположительных результатов при изменении динамических элементов, которые не влияют на общую функциональность интерфейса.

Способы маскирования

1. Ограничение области проверки. Можно задать область изображения, внутри которой выполняется сравнение. На примере видно, что сравнение изображений произошло в области заданного прямоугольника, а все остальное было проигнорировано.

2. Исключение области из проверки. Можно наоборот исключить часть изображения из сравнения. Например, сравнивалось все, кроме заданной области (мы исключили прямоугольную область на левом персонаже).

3. Обесцвечивание. В этом случае игнорируются различия цвета, но сохраняется проверка положения элементов. На примере игнорируется цвет одежды и волос, но сохраняется различие в положении волос и руки.

4. Подмена текста. В тестах, при создании сущностей в приложении, мы используем сгенерированные данные, которые от теста к тесту будут меняться. Поэтому перед созданием скриншота сгенерированный текст аккуратно заменяется фиксированным значением. Это позволяет стабилизировать результат.

В примере ниже имя пользователя меняется на фиксированное значение.

Вот как меняется реальный контент при маскировании в нашем продукте.

Слева — исходное всплывающее окно: аватары пользователей окрашиваются в случайный цвет, а логин меняется от теста к тесту. Справа — результат применения обесцвечивания и подмены сгенерированного текста.

5. Используем гибкие настройки сравнения

Попиксельное сравнение слишком строгое. Даже небольшое отличие приводит к падению теста. Чтобы повысить стабильность, можно:

- **установить допустимый коэффициент различия;**

Например, у нас допустимое отличие составляет 0,2%. Это позволяет игнорировать минимальные расхождения, которые не влияют на интерфейс.

- **игнорировать сглаживание границ цветов.**

В таком случае отдельные попиксельные отличия на границе перехода двух цветов не будут попадать в проверку.

Хранение эталонных скриншотов

Когда количество тестов выросло, появилась новая проблема — хранение эталонов. Сначала мы хранили изображения прямо в Git-репозитории. Для небольших проектов это вполне приемлемо. Но со временем возникли сложности.

Количество тестов увеличивалось, и вместе с ним росло число эталонных изображений. В итоге:

- около 3000 эталонных скриншотов в репозитории;
- размер репозитория вырос до 16,5 ГБ.

Это замедляло работу CI/CD, поскольку каждый запуск тестов требовал загрузки всего репозитория. Кроме того, частые обновления эталонов создавали шум в истории Git.

Решением проблемы стало использование Git LFS. Это расширение Git, предназначенное для хранения больших файлов.

После переноса изображений в отдельное хранилище, в репозитории сохраняются только указатели на файлы, что уменьшает размер репозитория во много раз.

При этом для нас процесс работы не изменился. При сохранении изменений в репозитории Git LFS автоматически загружает файлы в хранилище и заменяет их указателями. При получении изменений указатели автоматически заменяются обратно на реальные файлы.

Что мы получили после перехода на Git LFS:

- размер репозитория уменьшился с 16 ГБ до 190 МБ;
- сохранилась история изменений изображений репозитории;
- ускорилась работа CI/CD.

Совет простой: стоит заранее проанализировать ситуацию с хранением скриншотов и не допускать заполнение репозитория ненужными файлами.

Обновление эталонов

Ещё одна задача — поддержание эталонов в актуальном состоянии. Когда обновляется интерфейс, необходимо обновить соответствующие скриншоты.

Раньше алгоритм был следующий:

1. создать ветку;
2. запустить тест локально;
3. получить новый скриншот;
4. проверить его вручную;
5. сохранить изменения в репозитории.

Если нужно обновить один-два эталона — это быстро. Но при массовых изменениях интерфейса (например, при изменении дизайна) обновление десятков или сотен изображений занимало много времени.

Еще хотелось освободить тестировщиков от ручной работы и переключить на более приоритетные задачи.

Решением задачи стала возможность обновления скриншотов в CI/CD.

Мы перенесли этот процесс с локального компьютера в CI/CD. Для этого:

1. доработали функцию сравнения изображений. Теперь тесты могут работать в двух режимах:

- сравнение скриншотов;
- обновление эталонов.

2. добавили специальный флаг запуска, который будет говорить тестам, что нужно обновить скриншот вместо сравнения;

3. добавили шаг в CI/CD, который создаёт ветку, сохраняет обновлённые эталоны и отправляет изменения в репозиторий.

На тестировщике остаются две задачи: **оценка качества обновленных скриншотов и слияние с рабочей веткой**.

Можно выделить следующие преимущества такого подхода:

- рутина по обновлению эталонных скриншотов у нас происходит удаленно, рабочее место тестировщика не занято;
- сэкономленное время перераспределяется на приоритетные задачи;
- теперь обновлением эталонных скриншотов могут заниматься все заинтересованные лица, например, ручные тестировщики, без привлечения автоматизатора.

Заключение

Скриншотные тесты — мощный инструмент для обнаружения визуальных регрессий. Однако их эффективность напрямую зависит от того, насколько грамотно организована работа с ними.

Наш опыт показывает, что ключевыми факторами стабильности тестов являются:

- единое окружение для создания эталонов и выполнения тестов;
- исключение лишних элементов интерфейса;
- отключение анимаций;
- использование маскирования;
- гибкие правила сравнения изображений.

Эти практики не решают абсолютно всех проблем, но позволяют значительно повысить надёжность скриншотных тестов.

В любом случае подход к такому виду тестирования стоит адаптировать под особенности конкретного проекта.

Теги: [тестирование](#), [тесты](#), [qa](#), [автоматизация тестирования](#), [git lfs](#), [маскирование](#), [скриншотное тестирование](#), [надежные тесты](#), [стабильные тесты](#)

Хабы: [Блог компании iSpring](#), [Тестирование IT-систем](#), [Тестирование веб-сервисов](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



iSpring

Платформа для корпоративного обучения

[Сайт](#)



4

Карма

Павел Попов @pavelfargo

Пользователь

Подписаться



Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники



Простой



4 мин



16K



+96



30



27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно



Простой



11 мин



26K

Обзор

Перевод



+68



45



81



prohronus

18 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере



5 мин



9.2K

Кейс



+51



8



9



alizar

21 час назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой 7 мин 12K

Обзор

+37

59

18

beget_com
23 часа назад

Механический калькулятор. Как работает арифмометр?

Средний 10 мин 9.5K

Обзор

+37

23

8

TrexSelectel
19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

5 мин 8.7K

Обзор

+27

10

0

AlexSerbul
23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний 14 мин 9.9K

Мнение

+26

33

11

mariklolik
18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний 7 мин 7.4K

Кейс

+23

23

6

nick_dyuba
22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и ТіріТір

5 мин 7K

Recovery Mode

+22

8

5

Mimizavr
23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин 👁 6.9K

📌 +17

🔖 9

💬 7

Показать еще

ИНФОРМАЦИЯ

Сайт	www.ispring.ru
Дата регистрации	25 мая 2021
Дата основания	2001
Численность	201–500 человек
Местоположение	Россия
Представитель	Приёмко Андрей

ССЫЛКИ

[YouTube iSpring Dev](#)
youtube.com

[Как живут программисты в Йошкар-Оле](#)
www.ispring.ru

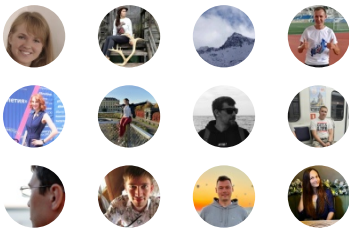
[О команде и вакансиях](#)
www.ispring.ru

ВКОНТАКТЕ



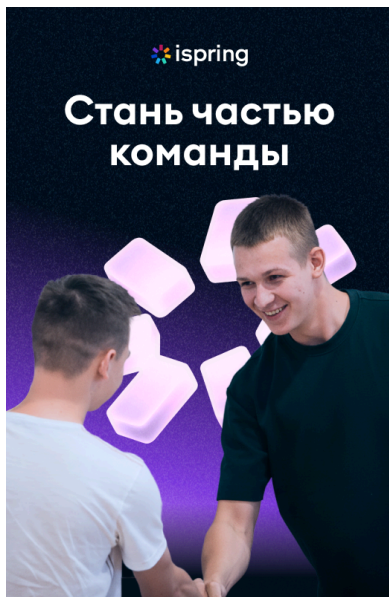
iSpring Dev

637 followers

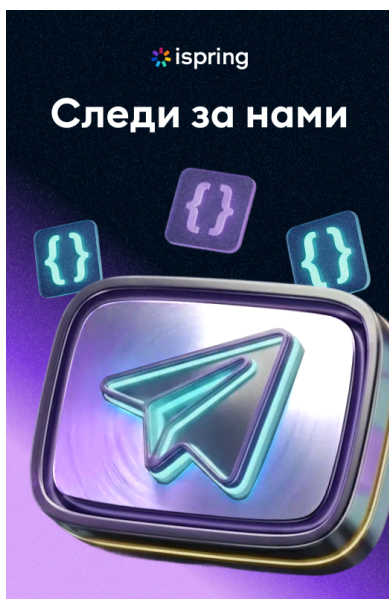


Follow on VK

ВИДЖЕТ



ВИДЖЕТ



БЛОГ НА ХАБРЕ

15 минут назад

Скриншотное тестирование: практические советы из реального проекта

607 0

1 апр в 01:31

Один вход для всех: как мы строили Gateway и выходили из хаоса nginx + Lua

8.2K 2

20 фев в 12:27

Проектирование системы отложенных уведомлений со склейкой: от требований до продакшена

14 ноя 2025 в 12:21

Интеграция OpenSearch: от функционального тестирования до проверки интеллекта поиска

12 сен 2025 в 22:20

Математика и веб-разработка: как мы добавили интерактивную кривую Безье в редактор изображений

Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам









Настройка языка

Техническая поддержка

© 2006–2026, Habr